

Тема урока: наследование

1. Процесс создания экземпляров класса
2. Наследование от неизменяемых типов данных (`int`, `float`, `str`, `tuple`)

Аннотация. Урок посвящен наследованию от встроенных неизменяемых типов данных в Python.

Процесс создания экземпляров класса

Напомним, что в Python процесс конструирования экземпляра класса состоит из двух шагов:

1. создание нового пустого экземпляра класса
2. инициализация созданного экземпляра класса

Для выполнения первого шага используется магический метод `__new__()`, который отвечает за создание и возврат нового пустого экземпляра класса. Затем созданный экземпляр передается методу `__init__()` для инициализации, то есть для добавления ему необходимых атрибутов.

Все классы в Python имеют магические методы `__new__()` и `__init__()`, так как являются дочерними классами класса `object` и наследуют от него данные методы.

Приведенный ниже код:

```
class Cat:
    pass

print('__new__' in dir(Cat))
print('__init__' in dir(Cat))
```

ВЫВОДИТ:

```
True
True
```

При создании практически каждого класса мы переопределяем метод `__init__()`, а метод `__new__()` оставляем исходным, так как у нас нет необходимости вмешиваться в процесс создания экземпляра класса, нам важно лишь получить готовый объект и наделить его нужными атрибутами.

Однако в некоторых задачах без переопределения метода `__new__()` не обойтись. Прежде чем перейти к решению одной из таких задач, сперва переопределим метод `__new__()`, повторим его стандартное поведение и поговорим о том, что происходит в его теле.

Приведенный ниже код:

```
class Cat:
    def __new__(cls, *args, **kwargs):
        print('1. Вызов метода __new__()')
        print(cls)
        return super().__new__(cls)

    def __init__(self, name):
        print('2. Вызов метода __init__()')
        self.name = name
        print(self)

    def __repr__(self):
        return f'Cat({repr(self.name)})'

cat = Cat('Кемаль')
```

ВЫВОДИТ:

```
1. Вызов метода __new__()
<class '__main__.Cat'>
2. Вызов метода __init__()
Cat('Кемаль')
```

Итак, мы имеем класс `Cat`, в котором определены методы `__new__()` и `__init__()`. В обоих методах содержится вывод некоторого текста, и порядок их вывода подтверждает сказанное в начале урока: сперва происходит создание экземпляра — вызов метода `__new__()`, а затем его инициализация — вызов метода `__init__()`.

Метод `__new__()` принимает один обязательный аргумент — класс, экземпляр которого требуется создать. Как можно заметить из вывода, этим аргументом всегда является сам класс, в котором реализован метод, в нашем случае это класс `Cat`. После указывается переменное количество позиционных и именованных аргументов, роль которых будет упомянута позже.

В теле метода `__new__()` нам важна последняя строка, в которой и происходит создание и возврат экземпляра класса. В ней мы с помощью функции `super()` обращаемся к родительскому методу `__new__()`, передавая в качестве аргумента текущий класс. Родительским классом является класс `object`, текущим — класс `Cat`, таким образом, вызов `super().__new__(cls)` равнозначен вызову `object.__new__(Cat)`.

Почему же нам нужно в нашем методе `__new__()` вызывать метод `__new__()` класса `object`? Дело в том, что именно метод `__new__()` класса `object` отвечает за создание всех объектов в Python. Он принимает единственный аргумент — класс, экземпляр которого требуется создать.

Именно этим мы и пользуемся: вызываем метод `__new__()` класса `object`, передаем ему в качестве аргумента наш класс `Cat` и получаем экземпляр класса `Cat`.



В Python все является объектом, даже сами классы. Так как за создание всех объектов отвечает метод `__new__()` класса `object`, все классы в Python являются экземплярами класса `object`.

Произвольное количество позиционных и именованных аргументов в методе `__new__()` необходимо для того чтобы не ограничивать сигнатуру метода `__init__()`, так как аргументы, передаваемые классу при создании его экземпляра, попадают как в метод `__new__()`, так и в метод `__init__()`.

Приведенный ниже код:

```
class Cat:
    def __new__(cls, *args, **kwargs):
        print(args, kwargs)
        return super().__new__(cls)

    def __init__(self, name, age=1):
        self.name = name
        self.age = age

cat = Cat('Кемаль', age=2)
```

ВЫВОДИТ:

```
('Кемаль',) {'age': 2}
```

Особенности поведения метода `__new__()`

Основная задача метода `__new__()` заключается в создании и возврате нового пустого экземпляра класса, который после будет передан в инициализатор. Однако метод `__new__()` может возвращать экземпляр класса, отличного от класса, реализующего сам метод. В таком случае метод `__init__()` вызываться не будет.

Приведенный ниже код:

```
class Animal:
    pass

class Cat:
    def __new__(cls, *args, **kwargs):
        print('1. Вызов метода __new__()')
        return Animal()

    def __init__(self, name):
        print('2. Вызов метода __init__()')
        self.name = name

cat = Cat('Кемаль')
```

ВЫВОДИТ:

```
1. Вызов метода __new__()
```

В примере выше метод `__new__()` класса `Cat` возвращает экземпляр другого класса — класса `Animal`, поэтому вызова метода `__init__()` класса `Cat` не происходит. Обратите внимание, что если методом `__new__()` будет возвращено значение `None`, то вызов метода `__init__()` так же не произойдет.

Особенности поведения метода `__init__()`

Основная задача метода `__init__()` заключается в наделении необходимыми атрибутами только что созданного пустого экземпляра класса. Важным является то, что возвращаемым значением метода всегда должен быть `None`, в противном случае будет возбуждено исключение.

Приведенный ниже код:

```
class Cat:
    def __new__(cls, *args, **kwargs):
        print('1. Создание экземпляра класса Cat')
        return super().__new__(cls)

    def __init__(self, name):
        print('2. Инициализация созданного экземпляра класса Cat')
        self.name = name
        return self

cat = Cat('Кемаль')
```

приводит к возбуждению исключения:

```
TypeError: __init__() should return None, not 'Cat'
```

Наследование от неизменяемых типов данных

При создании большинства классов мы не переопределяем метод `__new__()`, так как у нас нет необходимости вмешиваться в процесс создания экземпляра класса. Однако если нашей задачей будет создание класса, являющегося наследником какого-либо встроенного класса, экземпляры которого являются неизменяемыми, переопределение метода `__new__()` может нам очень помочь.

Переопределение метода `__new__()`

Предположим, мы хотим реализовать класс `Distance`, наследника класса `float`, описывающего расстояние на местности, который помимо своего числового значения имеет атрибут `unit`, в котором содержатся единицы измерения этого расстояния.

Если мы опустим переопределение метода `__new__()` и попытаемся решить задачу лишь с помощью метода `__init__()`, то столкнемся с некоторыми трудностями.

Приведенный ниже код:

```
class Distance(float):
    def __init__(self, value, unit):
        super().__init__()
        self.unit = unit

distance = Distance(1, 'Meters')

print(distance)
print(distance.unit)
```

приводит к возбуждению исключения:

```
TypeError: float expected at most 1 argument, got 2
```

Часть ошибки заключается в том, что при наследовании от неизменяемых типов данных их значение определяется при создании, то есть в методе `__new__()`, и менять его в инициализаторе уже поздно. Кроме того, выражение `Distance(1, 'Meters')` сперва вызывает метод `__new__()` класса `float`, который принимает лишь один аргумент — числовое или строковое значение, на основе которого будет создано число, а в нашем случае их передается два, что и приводит к ошибке.

Решить данную проблему можно путем расширения метода `__new__()`.

Приведенный ниже код:

```
class Distance(float):
    def __new__(cls, value, unit):
        instance = super().__new__(cls, value)
        instance.unit = unit
        return instance

distance = Distance(1, 'Meters')

print(distance)
print(distance.unit)
```

ВЫВОДИТ:

```
1.0
Meters
```

Сперва стоит обратить внимание на сигнатуру метода `__new__()`, он принимает три аргумента:

- `cls` — текущий класс, класс `Distance`
- `value` — числовое значение экземпляра класса `Distance`
- `unit` — единицы измерения, значение атрибута `unit` экземпляра класса `Distance`

В теле метода `__new__()` происходит вызов метода `__new__()` класса `float`, с обращением к нему при помощи функции `super()`. Таким образом, вызов `super().__new__(cls, value)` равнозначен вызову `float.__new__(Distance, value)`. Данный вызов возвращает экземпляр класса `Distance`, который имеет весь функционал класса `float`. Затем этому экземпляру добавляется атрибут `unit` с соответствующим значением, и он возвращается.

Важно отметить, что неизменяемой является лишь числовая составляющая экземпляра класса `Distance`, в то время как атрибут `unit` может быть изменен в любой части программы.

Приведенный ниже код:

```
class Distance(float):
    def __new__(cls, value, unit):
        instance = super().__new__(cls, value)
        instance.unit = unit
        return instance

distance = Distance(1, 'Meters')
print(distance, distance.unit)

distance.unit = 'Kilometers'
print(distance, distance.unit)
```

ВЫВОДИТ:

1.0 Meters
1.0 Kilometers

Дополнение класса

Более простой является ситуация, когда нас полностью устраивает функционал встроенного класса, однако мы хотим его расширить или изменить его, не затрагивая процесс создания экземпляра класса.

Примером такой ситуации может быть класс `WordCountString`, наследник класса `str`, который определяет дополнительный метод и переопределяет имеющийся. В качестве дополнительного будет метод `words()`, возвращающий количество слов в строке, которое вычисляется путем ее разделения по пробельным символам. В качестве переопределяемого будет метод `__str__()`, в котором помимо самой строки будет содержаться количество слов в ней.

Приведенный ниже код:

```
class WordCountString(str):
    def __str__(self):
        return f'{super().__str__()}, {self.words()}'

    def words(self):
        return len(self.split())

wordcountstring = WordCountString('I Love BeegEEK')

print(wordcountstring.words())
print(wordcountstring)
```

ВЫВОДИТ:

```
3
I Love BeegEEK, 3
```

Обратите внимание, что в нашем методе `__str__()` нам необходимо воспользоваться методом `__str__()` класса `str`. Если вместо `super().__str__()` мы укажем просто `self`, то получим бесконечные рекурсивные вызовы.

Примечания

Примечание 1. Наследование от встроенных классов ничем не отличается от наследования от собственных классов, то есть мы так же получаем все методы и атрибуты родительского класса.

Приведенный ниже код:

```
class Distance(float):
    def __new__(cls, value, unit):
        instance = super().__new__(cls, value)
        instance.unit = unit
        return instance
```

```
distance = Distance(1, 'Meters')
```

```
print(dir(distance))
```

ВЫВОДИТ:

```
['__abs__', '__add__', '__bool__', '__ceil__', '__class__', '__delattr__',
 'dict', '.', 'dir', '.', 'divmod', '.', 'doc', '.', 'ed', '.', 'float', '.', 'floor', '.']
```


